

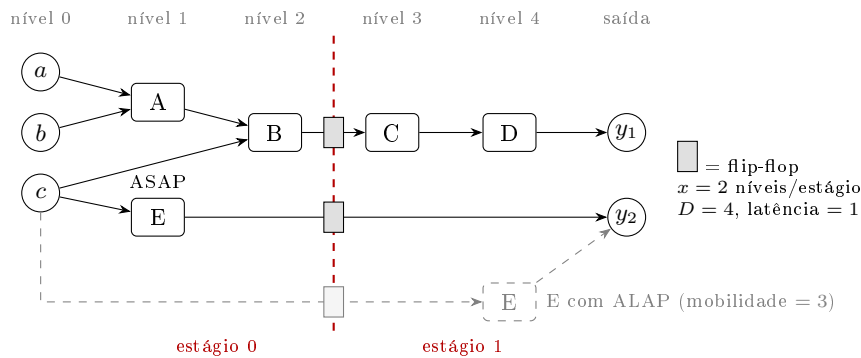
O que é. Em um circuito síncrono, o período de clock é limitado pelo caminho puramente combinacional mais lento entre dois registradores (o *caminho crítico*). O *retiming* (Leiserson & Saxe, 1991) move registradores através das células lógicas sem alterar o comportamento de entrada/saída do circuito, de modo que caminhos longos sejam cortados e os comprimentos dos caminhos fiquem balanceados. O *pipelining* é o caso particular usado aqui: a netlist de LUTs gerada pelo Yosys é puramente combinacional, então *adicionamos* $S - 1$ camadas de registradores e as posicionamos de forma que cada estágio tenha aproximadamente a mesma profundidade de LUTs. A função é preservada; as saídas apenas aparecem $S - 1$ ciclos depois.

Visão do circuito. Cada LUT conta como uma unidade de atraso (roteamento ignorado). Um caminho com D LUTs encadeadas custa $D \cdot t_{\text{LUT}}$, logo $f_{\text{max}} \approx 1/(D t_{\text{LUT}})$. Inserir uma camada de flip-flops a cada x níveis de LUT limita cada estágio a no máximo x LUTs, resultando em $f_{\text{max}} \approx 1/(x t_{\text{LUT}})$, ao custo de uma latência de $\lceil D/x \rceil - 1$ ciclos e de flip-flops extras. Para manter a corretude, *todo* caminho de uma entrada até uma saída deve atravessar o mesmo número de registradores; caso contrário, sinais de ciclos de clock diferentes seriam misturados.

Visão do grafo. O LUT2Graph gera um DAG $G = (V, E)$ com nós de entrada, nós de LUT e nós de saída; uma aresta (u, v) é uma net que sai de u e chega a um pino de entrada de v . O retiming é expresso por pesos nas arestas, $w(u, v)$ = número de registradores naquela aresta, e por um rótulo $s(v)$ por nó:

$$w_s(u, v) = w(u, v) + s(v) - s(u), \quad w(u, v) = 0 \text{ inicialmente}, \quad w_s(u, v) \geq 0.$$

Aqui $s(v)$ é o *estágio de pipeline* de v . Como s é não decrescente ao longo das arestas, nenhum peso fica negativo (retiming válido), e a soma dos pesos em qualquer caminho entrada→saída se reduz (soma telescópica) a $s(\text{saída}) - s(\text{entrada}) = S - 1$, ou seja, todos os caminhos têm a mesma latência.



Como a ferramenta faz (src/retime.py).

1. **Níveis ASAP:** em ordem topológica, as entradas recebem nível 0 e cada LUT recebe $1 + \max$ dos níveis de seus predecessores. A profundidade D é o maior nível (LUTs no caminho crítico).
2. **Níveis ALAP:** em ordem topológica reversa, cada LUT recebe o $\min$ dos níveis de seus sucessores -1 (as saídas ficam em $D + 1$). A *mobilidade* $\text{ALAP} - \text{ASAP}$ é a folga da LUT; LUTs críticas têm 0.
3. **Estágios:** os níveis são agrupados a cada x níveis (*-levels-per-stage*) ou em S faixas balanceadas (*-stages*). Cada LUT assume o estágio do seu nível ASAP ou ALAP (*-policy*). As entradas ficam no estágio 0 e as saídas são forçadas para o último estágio.
4. **Inserção de registradores:** uma aresta (u, v) recebe $s(v) - s(u)$ flip-flops. Registradores na mesma net são compartilhados como uma única cadeia de shift register, então uma net com fan-out paga apenas pelo destino mais distante.
5. **Exportação e verificação:** a netlist é reescrita em Yosys JSON/Verilog com células \$dff e uma porta clk, e comparada à original por simulação ciclo a ciclo com vetores aleatórios.

ASAP vs. ALAP. Só as LUTs com mobilidade > 0 mudam de lugar. O ASAP as puxa para o início, então os registradores ficam nas suas *saídas* (na figura: E no estágio 0, flip-flop em $E \rightarrow y_2$); o ALAP as empurra para o fim, então os registradores ficam nas suas *entradas* (E tracejada no estágio 1, flip-flop em $c \rightarrow E$). O caminho crítico e a latência são os mesmos; o que muda é o número de flip-flops e o balanceamento de LUTs por estágio, já que o compartilhamento depende de a net atrasada ter fan-out pequeno ou grande.

Escopo. Trata-se de pipelining baseado em níveis sobre um DAG, com atraso unitário por LUT. O retiming completo de Leiserson–Saxe (circuitos sequenciais com realimentação, atrasos reais, formulações de período mínimo ou de número mínimo de registradores) não é abordado.